ChatGPT conversation: Lab Submission Steps

Lab 4 (VibeCoding) · Vivek Varma · PES1UG24AM070 · Section 5B
Shared link: https://chatgpt.com/share/6ad1a30a-9cb4-83ee-8ce8-37464119acd1
Exported from the shared link on 16 Oct 2026

Uploaded a file Uploaded a file

hi, i have a software engineering lab called vibecoding. i'll paste the handout and the readme of the repo i got assigned (SETAPESU26/08_helicopter). first just tell me exactly what i have to submit and the order i should do things in so i dont miss anything

I've read both your Lab 4 handout and the `SETAPESU26/08_helicopter` README. The key is to complete the game fixes and features **in a controlled order, preserve evidence of the original broken game, and maintain separate Git commits for each task.**

The assignment is individual, has a stated duration of 60 minutes, and requires you to use a Vibe Coding tool such as ChatGPT or Claude as your debugging and pair-programming partner.

1. Exactly what you have to submit

Your final submission goes into your **personal Lab repository, inside a `Lab-4` folder**.

1. Before video — 10 seconds

Record the original game *before making any changes*. Show the broken behavior, ideally both the delayed reversal when switching directions and the helicopter flying off the bottom of the screen.

2. After video — 10 seconds

Show the corrected movement and boundaries, collision/game-over behavior, distance score, and shield activation and use. You may need to demonstrate several behaviors efficiently.

3. Updated code

Commit and push your completed game to your personal repository. Include the updated source files and any necessary project files.

4. AI conversation evidence

The handout asks for the chat history exported as a document/PDF. The README specifies the Chat/LLM page link with the complete chat history. To be safe, submit **both the shareable chat link and a complete exported PDF or document of the conversation**, if your tool supports both.

Important distinction: the handout lists videos, updated code, and an exported chat history as deliverables. The README's own checklist lists the two videos and the AI chat page link with the complete history. Including both forms of chat evidence avoids leaving that discrepancy unresolved.

The handout also explicitly requires:

- A separate Git commit for each task.
- Pushing the code to your personal repository.
- **No pull request to the main `SETAPESU26` repository.**

2. The exact order you should follow

Follow this sequence rather than immediately asking an AI to modify the entire repository.

01 **Set up your personal repository**

- Open the assigned repository: `SETAPESU26/08_helicopter`.
- Fork or clone it into your personal GitHub account, following your lab's expected workflow.
- Open the project in your chosen Vibe Coding tool or editor.
- Read the README and inspect the project files.

---

02 **Install dependencies and run the original game**

In the project terminal, run:

```bash
pip install -r requirements.txt
python main.py
```

The README requires Python 3.10 or newer. Confirm that the game launches and you can control the helicopter using the Up and Down arrow keys.

---

03 **Record your 10-second BEFORE video**

Do this before editing any source file. Demonstrate the original movement problem and missing bottom boundary. Save the recording somewhere safe.

04

Task 1 — Fix movement and boundaries

In `game/helicopter.py`:

- Cap the helicopter's vertical velocity.
- Make direction reversal responsive.
- Prevent the helicopter from leaving the screen at either edge.

Test these behaviors before committing.

Suggested commit: `Fix helicopter movement and boundaries`

05

Task 2 — Obstacle collision and game over

- Detect contact with either obstacle wall.
- End the game on an unprotected collision.
- Display a clear game-over message.
- Verify that safely passing through the gap does not trigger game over.

Suggested commit: `Add obstacle collision and game over`

06

Task 3 — Distance-based scoring

- Increase the score automatically as the helicopter travels.
- Display the score during gameplay.
- Display the final distance after game over.
- Reset the score to zero when a new game starts.

Suggested commit: `Add distance-based scoring`

07

Task 4 — Shield mechanic

- Let the player activate a shield.
- Clearly indicate when it is active.
- Absorb exactly one obstacle collision.
- Deactivate the shield immediately after absorbing the hit.
- Make the helicopter vulnerable again until the shield is activated again.

Suggested commit: `Add one-hit shield mechanic`

08

Run the complete regression test

Test the movement, both boundaries, collisions, safe passage, scoring, score reset, shield activation, shield absorption, and subsequent vulnerability. Fix any regressions and commit the fixes under the relevant task where practical.

09 **Record your 10-second AFTER video**

Record the completed game and demonstrate the new features. Check that the recording is saved and playable.

---

10 **Push code and prepare submission evidence**

- Push all commits to your personal repository.
- Export the complete AI conversation as a PDF/document.
- Obtain the shareable conversation link, if available.
- Upload the required files into `Lab-4` in your personal Lab repository.
- Verify the uploaded files and GitHub commits.

3. How to organize the submission

A clean structure would look like this:

```
Your-Personal-Lab-Repo/
└── Lab-4/
    ├── before.mp4
    ├── after.mp4
    ├── chat_history.pdf
    └── README.md
```

This is a suggested organization, not a prescribed folder layout from the handout. The `README.md` inside `Lab-4` can contain your AI conversation link. Your updated game source code should also be present in your personal GitHub repository; if your lab expects all deliverables inside `Lab-4`, place the relevant code there as well.

4. Final submission checklist

Lab 4 progress

0/15 complete

Before coding

- [ ] Fork or clone the assigned repository to my personal account

- [ ] Install dependencies and launch the original game
- [ ] Record and verify the 10-second before video

Implementation and Git

- [ ] Task 1: movement and boundary bug fixed
- [ ] Task 2: obstacle collision and game over implemented
- [ ] Task 3: distance score and reset implemented
- [ ] Task 4: one-hit shield implemented
- [ ] Test all expected behaviors
- [ ] Maintain separate commits for each task
- [ ] Push completed code to my personal repository

Submission

- [ ] Record and verify the 10-second after video
- [ ] Export the complete AI conversation as PDF/document
- [ ] Include the shareable AI chat link
- [ ] Upload deliverables under Lab-4
- [ ] Verify the submission and avoid opening a PR to SETAPESU26

Copy checklist

My recommendation: treat each of the four README tasks as a separate implementation-and-testing cycle, with its own commit. The most important thing not to forget is the **before video**: once you modify the code, you may not be able to reproduce the original behavior easily.

ok. im on windows and i already have a git repo called SE-Lab in my downloads with Lab-1 to Lab-3 folders. how do i get the helicopter code into a Lab-4 folder in that repo without messing up git? should i commit it before changing anything?

Yes — **commit your existing `SE-Lab` work before changing anything**, provided Lab-1 to Lab-3 are already in the state you want to preserve. Then put the helicopter project inside `Lab-4` and continue working there.

Since you're on Windows and already have a Git repository in Downloads, the safest approach is to use **your existing `SE-Lab` repository as the submission repository**, rather than accidentally creating a separate Git repository inside `Lab-4`.

The workflow we'll follow

1. Check that `SE-Lab` is a Git repository and that Lab-1 to Lab-3 are safe.
2. Commit any existing uncommitted work, if necessary.
3. Download the assigned helicopter repository into a temporary location.
4. Copy the helicopter project files into `SE-Lab\Lab-4` without copying the helicopter repository's `.git` folder.
5. Commit the original Lab-4 project before making any code changes.
6. Record the before video, then implement each task in a separate commit.

One important detail: **don't commit blindly if you have unrelated changes or files you don't want included.** We'll check Git's status first.

Step 1 — Inspect your existing repository

Open **PowerShell** (or the VS Code terminal) and run:

PowerShell

```
cd "$HOME\Downloads\SE-Lab"
git status
git branch --show-current
git remote -v
```

These commands show:

- Whether you have uncommitted changes.
- Which branch you're working on.
- Which GitHub repository your local folder is connected to.

What to do based on the result

- **If `git status` says `nothing to commit, working tree clean`**: your current work is already committed. You can proceed.
- **If it lists modified or untracked files:** inspect those changes first. If they are the intended Lab-1 to Lab-3 work, commit them before proceeding.

- **If you see unexpected changes:** don't run `git add .` or reset anything yet.

If you need to commit your existing work, and you've verified the changes are correct:

PowerShell

```
git add Lab-1 Lab-2 Lab-3
git commit -m "Save Labs 1-3 before Lab 4"
```

Only use that exact command if those are the actual folder names and the changes you want to preserve are inside them. If you have other intended changes, we should account for them separately.

Step 2 — Get the helicopter code ready

The assigned source repository is:

`https://github.com/SETAPESU26/08_helicopter`

Don't clone it directly into `SE-Lab\Lab-4` and then work inside it as a separate repository. That would create a nested `.git` directory, which can complicate submissions and commits.

Instead, clone it into a temporary sibling folder:

PowerShell

```
cd "$HOME\Downloads"
git clone https://github.com/SETAPESU26/08_helicopter.git helicopter-temp
```

If the repository is accessible, this creates:

```
Downloads/
├── SE-Lab/
│   ├── Lab-1/
│   ├── Lab-2/
│   └── Lab-3/
└── helicopter-temp/
    ├── .git/
    ├── main.py
    ├── requirements.txt
    ├── game/
    └── README.md
```

Step 3 — Copy the source into `SE-Lab\Lab-4`

First, ensure the destination doesn't already contain work you need to preserve:

```powershell
Test-Path "$HOME\Downloads\SE-Lab\Lab-4"
```

If it returns `False`, create the folder and copy the project while excluding `.git`:

```powershell
New-Item -ItemType Directory -Path "$HOME\Downloads\SE-Lab\Lab-4"
robocopy "$HOME\Downloads\helicopter-temp" "$HOME\Downloads\SE-Lab\Lab-4" /E /
```

This preserves the project files but excludes the source repository's Git metadata. Your `SE-Lab` repository remains the Git repository tracking the work.

If `Lab-4` already exists, stop before copying and inspect it so you don't overwrite existing work.

Step 4 — Verify and commit the original Lab-4 code

```powershell
cd "$HOME\Downloads\SE-Lab"
git status
git check-ignore -v Lab-4\main.py
```

The second command helps check whether the main source file is ignored by a Git ignore rule. If it prints a matching rule, investigate that before proceeding.

If everything looks right, stage the new folder:

```powershell
git add Lab-4
git status
```

Review the staged files. Make sure you see the expected project files and not unexpected temporary files, virtual environments, or other generated content.

Then commit the baseline:

```powershell
git commit -m "Add original helicopter game for Lab 4"
```

This is your clean starting point. You can now record the original game, make the four task-specific changes, and commit each task separately.

Do this first, before running the clone commands

Paste the output of these three commands here:

```powershell
cd "$HOME\Downloads\SE-Lab"
git status
git branch --show-current
git remote -v
```

I'll use that output to confirm your current Git state and tell you the safest next command. This avoids accidentally committing to the wrong repository or disturbing your existing labs.

how do i run the game?

Since you're on Windows, here's how to run your Helicopter game from your `SE-Lab` repository.

Step 1: Open the project in PowerShell

Run:

```powershell
cd "$HOME\Downloads\SE-Lab\Lab-4"
```

Check that you're in the correct folder:

```powershell
dir
```

You should see `main.py`, `requirements.txt`, the `game` folder, and `README.md`.

Step 2: Check Python

PowerShell

```
python --version
```

The lab README requires **Python 3.10 or newer**.

If `python` isn't recognized, try:

PowerShell

```
py --version
```

Step 3: Install the dependencies

PowerShell

```
python -m pip install -r requirements.txt
```

Wait for installation to finish. If you're using the `py` command instead, run:

PowerShell

```
py -m pip install -r requirements.txt
```

Step 4: Launch the game

PowerShell

```
python main.py
```

Or, if you're using the Python launcher:

PowerShell

```
py main.py
```

A game window should open. Use the **Up and Down arrow keys** to control the helicopter.

> **Before you modify the code**
>
> Record your original 10-second gameplay video now. Try holding Up and then immediately switching to Down to demonstrate the movement bug. Also try holding Down long enough to demonstrate the missing bottom boundary, if practical.

If the game doesn't launch

Send me the exact error message from PowerShell. Common issues include missing Pygame dependencies, an incorrect folder, or Python not being installed correctly. Don't modify the code just to fix an environment setup issue.

it runs and i can see the bugs the readme talks about. how do i record a 10 second clip of just the game window on windows, and what exactly should the before video show?

Use **Windows Snipping Tool's screen recording** if your version supports selecting a specific area. That lets you record just the Helicopter game window without showing your terminal or desktop.

1. How to record just the game window

Step 1 — Open Snipping Tool

Keep your Helicopter game running. Press the Windows key, search for `Snipping Tool`, and open it.

Step 2 — Select Record

Click the video camera/Record option, then click **New**. If prompted, drag a rectangle around the game window only.

Step 3 — Record for 10 seconds

Click Start, demonstrate the bug, wait around 10 seconds, then click Stop.

Step 4 — Save the video

Save it as `before.mp4` in an easy-to-find folder, such as `Downloads\SE-Lab\Lab-4`.

If your Snipping Tool doesn't offer recording or selecting the game area, use OBS Studio ↗ with a **Window Capture** source set to the Helicopter game. This is a good alternative for capturing only one application window.

2. What exactly should your BEFORE video show?

The README identifies two symptoms of the movement bug. Try to demonstrate both in one 10-second clip.

0–5s **Bug 1: Delayed direction reversal**

1. Hold the Up arrow for about 1–2 seconds.
2. Immediately switch to Down.
3. Keep the game visible as the helicopter continues drifting upward before descending.

This demonstrates that accumulated vertical velocity makes the controls feel unresponsive.

5–10s **Bug 2: Missing bottom boundary**

Hold the Down arrow and let the helicopter move beyond the bottom of the game window, if you can reproduce it within the remaining time.

This demonstrates that the original game doesn't stop the helicopter at the bottom edge.

If you cannot demonstrate both in 10 seconds: prioritize the delayed reversal and show the bottom-boundary bug in a second take if necessary. The handout requires a 10-second before video, so keep the final selected clip to 10 seconds.

You don't need to demonstrate collision, scoring, or shield behavior in this recording. Those are features you'll show in the AFTER video.

3. Quick check before moving on

- ○ The recording shows the actual game, not your terminal.
- ○ The original behavior is clearly visible.
- ○ The video is approximately 10 seconds long.
- ○ `before.mp4` is saved safely.
- ○ You haven't modified the original code yet.

Once the video is saved, you're ready to start Task 1: fixing movement and boundary handling.

lets do task 1 only. here's game/helicopter.py:

```
"""
Helicopter: the player-controlled vehicle. Moves vertically based on
held Up/Down keys.
"""

import pygame

THRUST = 0.4


class Helicopter:
    def __init__(self, x, y, width=40, height=24):
        self.x = x
        self.y = y
        self.width = width
        self.height = height
        self.vy = 0.0

    def handle_input(self, keys_pressed):
        if keys_pressed[pygame.K_UP]:
            self.vy -= THRUST
        if keys_pressed[pygame.K_DOWN]:
            self.vy += THRUST

    def update(self, height_bound):
        self.y += self.vy
        if self.y < 0:
            self.y = 0
            self.vy = 0
        # NOTE: no corresponding check against the bottom boundary

    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.width / 2), int(self.y - self.height / 2),
            self.width, self.height,
        )
```

explain why the helicopter keeps drifting up when i switch

from up to down, and why it can go off the bottom. then give me the fixed file. only change this file.

Show less

Task 1 — Fix helicopter movement and boundaries

There are two separate bugs in your current `game/helicopter.py`.

1. Why the helicopter keeps drifting upward

Your current input handler is:

```python
if keys_pressed[pygame.K_UP]:
    self.vy -= THRUST
if keys_pressed[pygame.K_DOWN]:
    self.vy += THRUST
```

Here, `vy` is the helicopter's vertical velocity, and `THRUST = 0.4`.

- Holding **Up** repeatedly subtracts `0.4` from `vy`, making it increasingly negative.
- Holding **Down** adds `0.4` per update, but only gradually cancels the accumulated negative velocity.
- The helicopter continues moving upward until `vy` reaches zero. Only after that can it begin descending.

For example, if `vy = -8`, switching to Down increases it by only `0.4` each update. It takes 20 updates just to reach zero, assuming one update per input-processing cycle.

The fix: limit the maximum vertical speed and make the opposite direction responsive, so excessive momentum cannot build up indefinitely.

2. Why it can fly off the bottom

Your `update()` method checks only this condition:

```python
if self.y < 0:
    self.y = 0
    self.vy = 0
```

There is no corresponding check for the bottom of the screen. Consequently, the helicopter's vertical position can keep increasing without a limit.

There's also a small geometry detail: `self.y` represents the helicopter's **center**, not its top edge. To keep the entire helicopter visible, the top limit should account for half its height, and the bottom limit should be `height_bound - self.height / 2`.

3. Fixed `game/helicopter.py`

Replace the entire contents of your existing file with the following. This changes **only** `game/helicopter.py` and preserves the existing class, method names, constructor, and rectangle interface.

Python

```python
"""
Helicopter: the player-controlled vehicle. Moves vertically based on
held Up/Down keys.
"""

import pygame

THRUST = 0.4
MAX_SPEED = 5.0


class Helicopter:
    def __init__(self, x, y, width=40, height=24):
        self.x = x
        self.y = y
        self.width = width
        self.height = height
        self.vy = 0.0

    def handle_input(self, keys_pressed):
        up_pressed = keys_pressed[pygame.K_UP]
        down_pressed = keys_pressed[pygame.K_DOWN]

        if up_pressed and not down_pressed:
            # Cancel downward momentum when reversing direction.
            if self.vy > 0:
                self.vy = 0.0

            self.vy = max(self.vy - THRUST, -MAX_SPEED)

        elif down_pressed and not up_pressed:
            # Cancel upward momentum when reversing direction.
            if self.vy < 0:
                self.vy = 0.0

            self.vy = min(self.vy + THRUST, MAX_SPEED)
```

```
    def update(self, height_bound):
        self.y += self.vy

        # Keep the entire helicopter inside the screen.
        min_y = self.height / 2
        max_y = max(min_y, height_bound - self.height / 2)

        if self.y < min_y:
            self.y = min_y
            self.vy = 0.0
        elif self.y > max_y:
            self.y = max_y
            self.vy = 0.0

    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.width / 2), int(self.y - self.height / 2),
            self.width, self.height,
        )
```

4. What changed and why

Change	Purpose
`MAX_SPEED = 5.0`	Prevents unlimited vertical velocity.
Opposite-direction velocity cancellation	Removes old momentum when the player reverses direction.
`min_y = self.height / 2`	Keeps the helicopter's top edge inside the screen.
`max_y = height_bound - self.height / 2`	Keeps its bottom edge inside the screen.
Reset `vy` at either boundary	Prevents accumulated velocity from pushing it beyond the boundary on subsequent frames.

The reversal now starts responding in the opposite direction immediately, although acceleration still determines how quickly the helicopter builds speed in that direction.

5. Test Task 1 before committing

Save the file and restart the game:

PowerShell

```
python main.py
```

Verify each behavior:

- [ ] Hold Up, then switch to Down. The helicopter should reverse responsively.
- [ ] Hold Down for several seconds. The helicopter must remain visible at the bottom.
- [ ] Hold Up for several seconds. The helicopter must remain visible at the top.
- [ ] Hold either direction continuously. Speed should remain capped.
- [ ] Release both keys and check that the game still behaves as expected.

Important: don't implement collision detection, scoring, or the shield yet. Those are separate tasks.

If all tests pass, save Task 1 in its own commit:

PowerShell

```
git add Lab-4/game/helicopter.py
git commit -m "Fix helicopter movement and boundaries"
```

Run those Git commands from your `SE-Lab` repository root. Before committing, use `git status` to ensure you're not accidentally including unrelated changes.

better, but when i hold up half the helicopter still goes above the screen, and switching direction still has a small delay. can you fix both?

The two issues likely come from different places:

1. **The helicopter clips above the screen:** the boundary calculation needs to account for the helicopter's full rectangle, and the renderer's coordinate convention may also matter. Your `get_rect()` uses `self.y` as the center, so the correct top limit is half the helicopter's height.
2. **Direction reversal still feels delayed:** the previous fix clears the old velocity, but then applies only `THRUST = 0.4` toward the new direction. That means the helicopter starts reversing immediately, but builds up speed gradually.

I'll keep the fix limited to `game/helicopter.py`, as requested. One important distinction: if the helicopter is still visually clipping after the boundary fix, we should verify how the renderer positions it rather than changing other files without permission.

Replace the file with this revised version. It adds a more conservative boundary margin and explicitly sets velocity in the new direction when the player reverses.

Python

```python
"""
Helicopter: the player-controlled vehicle. Moves vertically based on
held Up/Down keys.
"""

import pygame

THRUST = 0.4
MAX_SPEED = 5.0


class Helicopter:
    def __init__(self, x, y, width=40, height=24):
        self.x = x
        self.y = y
        self.width = width
        self.height = height
        self.vy = 0.0

    def handle_input(self, keys_pressed):
        up_pressed = keys_pressed[pygame.K_UP]
        down_pressed = keys_pressed[pygame.K_DOWN]

        if up_pressed and not down_pressed:
            if self.vy > 0:
                # Reverse immediately instead of first cancelling momentum.
                self.vy = -THRUST
            else:
                self.vy = max(self.vy - THRUST, -MAX_SPEED)

        elif down_pressed and not up_pressed:
            if self.vy < 0:
                # Reverse immediately instead of first cancelling momentum.
                self.vy = THRUST
            else:
                self.vy = min(self.vy + THRUST, MAX_SPEED)

    def update(self, height_bound):
        self.y += self.vy

        # Use a conservative margin to keep the helicopter fully visible.
        min_y = self.height
        max_y = max(min_y, height_bound - self.height)

        if self.y < min_y:
```

```
            self.y = min_y
            self.vy = 0.0
        elif self.y > max_y:
            self.y = max_y
            self.vy = 0.0

    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.width / 2), int(self.y - self.height / 2),
            self.width, self.height,
        )
```

Retest

1. Save the file and restart `python main.py`.
2. Hold Up, then switch directly to Down several times. The velocity should change direction immediately.
3. Hold Up against the top boundary and Down against the bottom boundary. The helicopter should remain fully visible.

Note: this uses a more conservative boundary than the geometric minimum. If the sprite still clips, the renderer may draw the helicopter differently from the rectangle returned by `get_rect()`. We would need to inspect `game/renderer.py` to confirm that, but I haven't changed that file.

Don't commit this revision until both behaviors pass your tests.

works now. what git commands do i use to commit just this task, and can you write a short commit message for it?

Run these commands in PowerShell to commit **Task 1 only**.

1. Go to your repository root

PowerShell

```
cd "$HOME\Downloads\SE-Lab"
```

2. Check your changes

PowerShell

```
git status
```

3. Stage only the helicopter file

PowerShell

```
git add Lab-4/game/helicopter.py
```

This stages only the file you changed, leaving other files untouched.

4. Verify what's staged

PowerShell

```
git diff --cached
```

Confirm the changes shown are only your Task 1 movement and boundary fixes.

5. Commit Task 1

Recommended commit message

`Fix helicopter movement and boundaries` Copy

Run:

PowerShell

```
git commit -m "Fix helicopter movement and boundaries"
```

6. Verify the commit

PowerShell

```
git status
git log -1 --oneline
```

If you have no other uncommitted changes, Git should report a clean working tree.

Don't push yet unless you want to. You can complete Tasks 2, 3, and 4 as separate commits, then push all your commits together at the end.

task 2 now. here are my current files:

game_engine.py:
"""
GameEngine: owns the helicopter and all obstacles.

Starter version: the helicopter moves and obstacles scroll by, but
there's no collision detection at all yet (the helicopter can fly straight through obstacles harmlessly), no scoring, and no shield.
That's Tasks 2, 3, and 4. Movement also has known bugs (see
game/helicopter.py) that Task 1 asks you to fix.
"""

```
import random

from game.helicopter import Helicopter
from game.obstacle import Obstacle
from game.renderer import WIDTH, HEIGHT

SPAWN_INTERVAL_FRAMES = 90
GAP_HEIGHT = 150
WALL_WIDTH = 60
SCROLL_SPEED = 3


class GameEngine:
    def __init__(self):
        self.helicopter = Helicopter(x=100, y=HEIGHT / 2)
        self.obstacles = []
        self.frames_until_spawn = 0

    def _spawn_obstacle(self):
        margin = 60
        gap_y = random.randint(margin + GAP_HEIGHT // 2,
HEIGHT - margin - GAP_HEIGHT // 2)
        self.obstacles.append(Obstacle(
            x=WIDTH, gap_y=gap_y, gap_height=GAP_HEIGHT,
```

```
            wall_width=WALL_WIDTH, screen_height=HEIGHT,
speed=SCROLL_SPEED,
        ))

    def handle_input(self, keys_pressed):
        self.helicopter.handle_input(keys_pressed)

    def handle_keydown(self, key):
        pass

    def update(self):
        self.helicopter.update(HEIGHT)

        self.frames_until_spawn -= 1
        if self.frames_until_spawn <= 0:
            self._spawn_obstacle()
            self.frames_until_spawn =
SPAWN_INTERVAL_FRAMES

        for obstacle in self.obstacles:
            obstacle.update()
        self.obstacles = [o for o in self.obstacles if not
o.is_off_screen()]

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_scene(surface, self.helicopter,
self.obstacles)
obstacle.py:
"""
Obstacle: a scrolling wall pair with a gap the helicopter must
fly
through.
"""

import pygame


class Obstacle:
    def __init__(self, x, gap_y, gap_height, wall_width,
screen_height, speed):
        self.x = x
```

```
        self.gap_y = gap_y
        self.gap_height = gap_height
        self.wall_width = wall_width
        self.screen_height = screen_height
        self.speed = speed
        self.scored = False   # used for distance/pass tracking
later

    def update(self):
        self.x -= self.speed

    def is_off_screen(self):
        return self.x + self.wall_width < 0

    def get_top_rect(self):
        top_height = self.gap_y - self.gap_height / 2
        return pygame.Rect(int(self.x), 0, self.wall_width,
int(top_height))

    def get_bottom_rect(self):
        bottom_y = self.gap_y + self.gap_height / 2
        return pygame.Rect(int(self.x), int(bottom_y),
self.wall_width, int(self.screen_height - bottom_y))
renderer.py:
"""
renderer: all pygame drawing lives here, kept separate from
game logic.
"""

import pygame

WIDTH, HEIGHT = 700, 500
WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_BG = (140, 200, 230)
COLOR_HELI = (60, 60, 70)
COLOR_OBSTACLE = (70, 150, 80)
COLOR_TEXT = (20, 20, 20)



def draw_scene(surface, helicopter, obstacles):
    surface.fill(COLOR_BG)
```

```
    for obstacle in obstacles:
        pygame.draw.rect(surface, COLOR_OBSTACLE,
obstacle.get_top_rect())
        pygame.draw.rect(surface, COLOR_OBSTACLE,
obstacle.get_bottom_rect())
    pygame.draw.rect(surface, COLOR_HELI,
helicopter.get_rect(), border_radius=4)


def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)


def draw_banner(surface, font, text):
    surf = font.render(text, True, (180, 40, 40))
    rect = surf.get_rect(center=(surface.get_width() // 2,
surface.get_height() // 2))
    surface.blit(surf, rect)
```

main.py:

```
"""
Helicopter Game (Lab Starter)

Run with:  python3 main.py

Controls: Up/Down arrows to move.
"""

import pygame

from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE


def main():
    pygame.init()
    screen = pygame.display.set_mode(WINDOW_SIZE)
    pygame.display.set_caption("Helicopter")
    clock = pygame.time.Clock()
    font = pygame.font.SysFont("consolas", 22)

    engine = GameEngine()
    running = True
```

```
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.KEYDOWN:
                engine.handle_keydown(event.key)

        keys = pygame.key.get_pressed()
        engine.handle_input(keys)
        engine.update()
        engine.draw(screen, font)

        pygame.display.flip()
        clock.tick(60)

    pygame.quit()


if __name__ == "__main__":
    main()
```

add collision so touching the top or bottom wall ends the game and shows a clear game over message. flying through the gap should never end it. also add R to restart after game over, because task 3 needs the score to reset on a new game.

Show less

Task 2 — Obstacle collision and game over

We'll implement collision detection using the existing `pygame.Rect` objects, so the helicopter only crashes when its rectangle actually touches a wall. Passing through the gap should remain safe.

Your request also includes an **R-to-restart control**. That's a sensible addition now because Task 3 requires the score to reset when a new game starts. We'll implement the restart mechanism without adding scoring yet.

How collision detection will work

- Get the helicopter's rectangle using `helicopter.get_rect()`.

- For every obstacle, check its top and bottom wall rectangles using `get_top_rect()` and `get_bottom_rect()`.
- If the helicopter overlaps either wall, set the game to a game-over state.
- Display a clear game-over message and stop normal gameplay updates until the player presses **R**.
- On restart, reset the helicopter and obstacles to their initial state.

One important detail: collision must be checked against both wall rectangles, **not the obstacle's entire width and height**, because that would incorrectly treat the safe gap as a collision.

1. Replace `game/game_engine.py`

This version adds the game-over state, collision checks, and restart logic while leaving scoring and shield functionality for their respective tasks.

Python

```python
"""
GameEngine: owns the helicopter and all obstacles.
"""

import random

import pygame

from game.helicopter import Helicopter
from game.obstacle import Obstacle
from game.renderer import WIDTH, HEIGHT

SPAWN_INTERVAL_FRAMES = 90
GAP_HEIGHT = 150
WALL_WIDTH = 60
SCROLL_SPEED = 3


class GameEngine:
    def __init__(self):
        self.reset()

    def reset(self):
        """Start a fresh game."""
        self.helicopter = Helicopter(x=100, y=HEIGHT / 2)
        self.obstacles = []
        self.frames_until_spawn = 0
        self.game_over = False
```

```
    def _spawn_obstacle(self):
        margin = 60
        gap_y = random.randint(
            margin + GAP_HEIGHT // 2,
            HEIGHT - margin - GAP_HEIGHT // 2,
        )
        self.obstacles.append(
            Obstacle(
                x=WIDTH,
                gap_y=gap_y,
                gap_height=GAP_HEIGHT,
                wall_width=WALL_WIDTH,
                screen_height=HEIGHT,
                speed=SCROLL_SPEED,
            )
        )

    def handle_input(self, keys_pressed):
        if not self.game_over:
            self.helicopter.handle_input(keys_pressed)

    def handle_keydown(self, key):
        if self.game_over and key == pygame.K_r:
            self.reset()

    def _check_collisions(self):
        helicopter_rect = self.helicopter.get_rect()

        for obstacle in self.obstacles:
            if (
                helicopter_rect.colliderect(obstacle.get_top_rect())
                or helicopter_rect.colliderect(obstacle.get_bottom_rect())
            ):
                self.game_over = True
                return

    def update(self):
        # Freeze gameplay after a collision.
        if self.game_over:
            return

        self.helicopter.update(HEIGHT)

        self.frames_until_spawn -= 1
        if self.frames_until_spawn <= 0:
            self._spawn_obstacle()
            self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

        for obstacle in self.obstacles:
            obstacle.update()
```

```
        self.obstacles = [
            obstacle
            for obstacle in self.obstacles
            if not obstacle.is_off_screen()
        ]

        self._check_collisions()

    def draw(self, surface, font):
        from game import renderer

        renderer.draw_scene(surface, self.helicopter, self.obstacles)

        if self.game_over:
            renderer.draw_banner(
                surface, font, "GAME OVER - Press R to restart"
            )
```

2. Do you need to change `main.py`, `obstacle.py`, or `renderer.py`?

No. Your existing files already provide everything needed:

- `main.py` forwards key presses to `handle_keydown()`.
- `obstacle.py` exposes the top and bottom wall rectangles.
- `renderer.py` already has `draw_banner()` for the game-over message.

The updated engine reuses these interfaces, so **only `game/game_engine.py` needs to change for Task 2**.

3. Test Task 2

Save the file and run:

PowerShell

```
python main.py
```

Test the following:

- [ ] The game launches normally.
- [ ] Touching a top obstacle wall triggers game over.
- [ ] Touching a bottom obstacle wall triggers game over.
- [ ] Flying safely through the gap does not trigger game over.
- [ ] Obstacles and helicopter movement freeze after game over.
- [ ] Pressing `R` restarts the game with a new helicopter and fresh obstacles.
- [ ] The game continues normally after restarting.

If you can, test both collisions independently. A collision near the edge of the gap is especially useful for confirming the rectangles are handled correctly.

4. Commit Task 2 separately

Once the tests pass, run these commands from your `SE-Lab` repository root:

PowerShell

```
git status
git add Lab-4/game/game_engine.py
git diff --cached
git commit -m "Add obstacle collision and game over"
```

The commit includes only `game_engine.py`, leaving your Task 1 commit intact.

Scope note: Restarting currently resets the game state and prepares the engine for Task 3's score reset. We haven't implemented scoring, shield behavior, or a separate restart prompt yet; the game-over banner already tells the player to press R.

after game over the walls keep moving behind the message and i can still move the helicopter. it should freeze until i press R.

The issue is that `game_engine.py` freezes updates after game over, but `main.py` still calls `engine.handle_input(keys)` every frame. Your current `handle_input()` should block movement, so let's make the freeze more robust by guarding both input and updates, and ensuring the game-over state is set immediately when a collision occurs.

In `game/game_engine.py`, make sure these methods match the following:

Python

```
def handle_input(self, keys_pressed):
    if self.game_over:
        return

    self.helicopter.handle_input(keys_pressed)


def handle_keydown(self, key):
```

```
        if self.game_over and key == pygame.K_r:
            self.reset()


    def update(self):
        if self.game_over:
            return

        self.helicopter.update(HEIGHT)

        self.frames_until_spawn -= 1
        if self.frames_until_spawn <= 0:
            self._spawn_obstacle()
            self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

        for obstacle in self.obstacles:
            obstacle.update()

        self.obstacles = [
            obstacle
            for obstacle in self.obstacles
            if not obstacle.is_off_screen()
        ]

        self._check_collisions()
```

Also verify that `_check_collisions()` sets `self.game_over = True` when either wall is hit.

Why this should fix it

- **Walls moving:** `update()` returns immediately after game over, before obstacles move.
- **Helicopter moving:** `handle_input()` returns immediately after game over, so key holds cannot affect its velocity.
- **Restart:** pressing `R` calls `reset()`, which initializes a fresh game and clears `game_over`.

Save the file and run `python main.py` again. After a collision, neither the helicopter nor the walls should move until you press `R`.

If the walls still move, send me your current complete `game_engine.py` so I can identify what's different from this version.

task 3: distance score. it should go up by itself while playing, show the final distance on the game over screen, and reset to 0 when i press R. give my current game_engine.py

Replace your current `game/game_engine.py` with the version below. It keeps **Task 1 movement fixes** and **Task 2 collision/game-over/restart functionality**, and adds **Task 3 distance scoring**.

Distance increases automatically while playing, stops when the game ends, appears on the game-over screen, and resets to `0` when you press **R**.

Python

```python
"""
GameEngine: owns the helicopter and all obstacles.
Handles movement, collisions, distance scoring, and restarting.
"""

import random
import pygame

from game.helicopter import Helicopter
from game.obstacle import Obstacle
from game.renderer import WIDTH, HEIGHT

SPAWN_INTERVAL_FRAMES = 90
GAP_HEIGHT = 150
WALL_WIDTH = 60
SCROLL_SPEED = 3


class GameEngine:
    def __init__(self):
        self.reset()

    def reset(self):
        """Start a fresh game."""
        self.helicopter = Helicopter(x=100, y=HEIGHT / 2)
        self.obstacles = []
        self.frames_until_spawn = 0
        self.game_over = False
        self.distance = 0

    def _spawn_obstacle(self):
        margin = 60
```

```
        gap_y = random.randint(
            margin + GAP_HEIGHT // 2,
            HEIGHT - margin - GAP_HEIGHT // 2,
        )

        self.obstacles.append(
            Obstacle(
                x=WIDTH,
                gap_y=gap_y,
                gap_height=GAP_HEIGHT,
                wall_width=WALL_WIDTH,
                screen_height=HEIGHT,
                speed=SCROLL_SPEED,
            )
        )

    def handle_input(self, keys_pressed):
        if not self.game_over:
            self.helicopter.handle_input(keys_pressed)

    def handle_keydown(self, key):
        if self.game_over and key == pygame.K_r:
            self.reset()

    def _check_collisions(self):
        helicopter_rect = self.helicopter.get_rect()

        for obstacle in self.obstacles:
            if (
                helicopter_rect.colliderect(obstacle.get_top_rect())
                or helicopter_rect.colliderect(obstacle.get_bottom_rect())
            ):
                self.game_over = True
                return

    def update(self):
        # Freeze the game once a collision occurs.
        if self.game_over:
            return

        self.helicopter.update(HEIGHT)

        self.frames_until_spawn -= 1
        if self.frames_until_spawn <= 0:
            self._spawn_obstacle()
            self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

        for obstacle in self.obstacles:
            obstacle.update()

        self.obstacles = [
```

```
            obstacle
            for obstacle in self.obstacles
            if not obstacle.is_off_screen()
        ]

        # Task 3: increase distance automatically while playing.
        self.distance += SCROLL_SPEED

        self._check_collisions()

    def draw(self, surface, font):
        from game import renderer

        renderer.draw_scene(surface, self.helicopter, self.obstacles)

        # Display the current distance during gameplay.
        renderer.draw_text(
            surface,
            font,
            f"Distance: {self.distance}",
            (10, 10),
        )

        if self.game_over:
            renderer.draw_banner(
                surface,
                font,
                f"GAME OVER - Distance: {self.distance} - Press R to restart",
            )
```

Test Task 3

1. Save `game/game_engine.py`.
2. Close the running game window completely and launch it again:

   ‹/› PowerShell
   ```
   python main.py
   ```
3. While playing, verify that the distance increases automatically.
4. Crash into a wall. Verify that the distance stops increasing and the final distance appears on the game-over screen.
5. Press **R**. Verify that the game restarts and the distance returns to `0`.

Note: Distance is measured in game-distance units, increasing by `3` per frame at the current scroll speed. It is not a real-world distance measurement. The game-over banner

may be long, so if the text extends beyond the window, we can adjust the display without changing the scoring logic.

last one, task 4. i want S to turn on a shield. it should block one hit, show clearly that its on, and turn off the moment it takes a hit. after that im vulnerable again until i press S again. current files:

game_engine.py:

```
"""
GameEngine: owns the helicopter and all obstacles.
Handles movement, collisions, distance scoring, and
restarting.
"""

import random
import pygame

from game.helicopter import Helicopter
from game.obstacle import Obstacle
from game.renderer import WIDTH, HEIGHT

SPAWN_INTERVAL_FRAMES = 90
GAP_HEIGHT = 150
WALL_WIDTH = 60
SCROLL_SPEED = 3


class GameEngine:
    def __init__(self):
        self.reset()

    def reset(self):
        """Start a fresh game."""
        self.helicopter = Helicopter(x=100, y=HEIGHT / 2)
        self.obstacles = []
        self.frames_until_spawn = 0
```

```
        self.game_over = False
        self.distance = 0

    def _spawn_obstacle(self):
        margin = 60
        gap_y = random.randint(
            margin + GAP_HEIGHT // 2,
            HEIGHT - margin - GAP_HEIGHT // 2,
        )

        self.obstacles.append(
            Obstacle(
                x=WIDTH,
                gap_y=gap_y,
                gap_height=GAP_HEIGHT,
                wall_width=WALL_WIDTH,
                screen_height=HEIGHT,
                speed=SCROLL_SPEED,
            )
        )

    def handle_input(self, keys_pressed):
        if not self.game_over:
            self.helicopter.handle_input(keys_pressed)

    def handle_keydown(self, key):
        if self.game_over and key == pygame.K_r:
            self.reset()

    def _check_collisions(self):
        helicopter_rect = self.helicopter.get_rect()

        for obstacle in self.obstacles:
            if (
                helicopter_rect.colliderect(obstacle.get_top_rect())
                or
helicopter_rect.colliderect(obstacle.get_bottom_rect())
            ):
                self.game_over = True
                return

    def update(self):
```

```
        # Freeze the game once a collision occurs.
        if self.game_over:
            return

        self.helicopter.update(HEIGHT)

        self.frames_until_spawn -= 1
        if self.frames_until_spawn <= 0:
            self._spawn_obstacle()
            self.frames_until_spawn =
SPAWN_INTERVAL_FRAMES

        for obstacle in self.obstacles:
            obstacle.update()

        self.obstacles = [
            obstacle
            for obstacle in self.obstacles
            if not obstacle.is_off_screen()
        ]

        # Task 3: increase distance automatically while playing.
        self.distance += SCROLL_SPEED

        self._check_collisions()

    def draw(self, surface, font):
        from game import renderer

        renderer.draw_scene(surface, self.helicopter,
self.obstacles)

        # Display the current distance during gameplay.
        renderer.draw_text(
            surface,
            font,
            f"Distance: {self.distance}",
            (10, 10),
        )

        if self.game_over:
            renderer.draw_banner(
```

```
            surface,
            font,
            f"GAME OVER - Distance: {self.distance} - Press R
to restart",
        )

renderer.py:
"""
renderer: all pygame drawing lives here, kept separate from
game logic.
"""

import pygame

WIDTH, HEIGHT = 700, 500
WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_BG = (140, 200, 230)
COLOR_HELI = (60, 60, 70)
COLOR_OBSTACLE = (70, 150, 80)
COLOR_TEXT = (20, 20, 20)


def draw_scene(surface, helicopter, obstacles):
    surface.fill(COLOR_BG)
    for obstacle in obstacles:
        pygame.draw.rect(surface, COLOR_OBSTACLE,
obstacle.get_top_rect())
        pygame.draw.rect(surface, COLOR_OBSTACLE,
obstacle.get_bottom_rect())
    pygame.draw.rect(surface, COLOR_HELI,
helicopter.get_rect(), border_radius=4)


def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)


def draw_banner(surface, font, text):
    surf = font.render(text, True, (180, 40, 40))
    rect = surf.get_rect(center=(surface.get_width() // 2,
```

```
surface.get_height() // 2))
    surface.blit(surf, rect)
```

Show less

For Task 4, you only need to modify `game_engine.py` **and** `renderer.py`. This implementation preserves Tasks 1–3 and adds a manually activated, one-hit shield.

- Press **S** to activate the shield.
- The shield is clearly shown around the helicopter, with an on-screen status.
- The first obstacle collision consumes the shield and turns it off without ending the game.
- You are vulnerable again immediately afterward. Press **S** to reactivate it.
- If you collide while the shield is off, the game ends.
- Press **R** after game over to restart with distance `0` and the shield off.

1. Replace `game/game_engine.py`

Python

```python
"""
GameEngine: owns the helicopter and all obstacles.
Handles movement, collisions, distance scoring, and the one-hit shield.
"""

import random
import pygame

from game.helicopter import Helicopter
from game.obstacle import Obstacle
from game.renderer import WIDTH, HEIGHT

SPAWN_INTERVAL_FRAMES = 90
GAP_HEIGHT = 150
WALL_WIDTH = 60
SCROLL_SPEED = 3


class GameEngine:
    def __init__(self):
        self.reset()

    def reset(self):
        """Start a fresh game."""
        self.helicopter = Helicopter(x=100, y=HEIGHT / 2)
        self.obstacles = []
        self.frames_until_spawn = 0
```

```
        self.game_over = False
        self.distance = 0
        self.shield_active = False

    def _spawn_obstacle(self):
        margin = 60
        gap_y = random.randint(
            margin + GAP_HEIGHT // 2,
            HEIGHT - margin - GAP_HEIGHT // 2,
        )

        self.obstacles.append(
            Obstacle(
                x=WIDTH,
                gap_y=gap_y,
                gap_height=GAP_HEIGHT,
                wall_width=WALL_WIDTH,
                screen_height=HEIGHT,
                speed=SCROLL_SPEED,
            )
        )

    def handle_input(self, keys_pressed):
        if not self.game_over:
            self.helicopter.handle_input(keys_pressed)

    def handle_keydown(self, key):
        if self.game_over:
            if key == pygame.K_r:
                self.reset()
            return

        # Toggle the shield with S while the game is active.
        if key == pygame.K_s:
            self.shield_active = not self.shield_active

    def _check_collisions(self):
        helicopter_rect = self.helicopter.get_rect()

        for obstacle in self.obstacles:
            collided = (
                helicopter_rect.colliderect(obstacle.get_top_rect())
                or helicopter_rect.colliderect(obstacle.get_bottom_rect())
            )

            if not collided:
                continue

            if self.shield_active:
                # Absorb one hit and immediately deactivate the shield.
                self.shield_active = False
```

```
                return

            # No shield: collision ends the game.
            self.game_over = True
            return

    def update(self):
        # Freeze gameplay after game over.
        if self.game_over:
            return

        self.helicopter.update(HEIGHT)

        self.frames_until_spawn -= 1
        if self.frames_until_spawn <= 0:
            self._spawn_obstacle()
            self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

        for obstacle in self.obstacles:
            obstacle.update()

        self.obstacles = [
            obstacle
            for obstacle in self.obstacles
            if not obstacle.is_off_screen()
        ]

        # Task 3: increase distance automatically during gameplay.
        self.distance += SCROLL_SPEED

        # Task 2 and Task 4: check collisions and handle shield protection.
        self._check_collisions()

    def draw(self, surface, font):
        from game import renderer

        renderer.draw_scene(
            surface,
            self.helicopter,
            self.obstacles,
            shield_active=self.shield_active,
        )

        renderer.draw_text(
            surface,
            font,
            f"Distance: {self.distance}",
            (10, 10),
        )

        shield_status = (
```

```
            "SHIELD: ON (S to deactivate)"
            if self.shield_active
            else "SHIELD: OFF (S to activate)"
        )

        status_color = (
            (0, 100, 220) if self.shield_active else (80, 50, 50)
        )

        renderer.draw_text(
            surface,
            font,
            shield_status,
            (10, 40),
            color=status_color,
        )

        if self.game_over:
            renderer.draw_banner(
                surface,
                font,
                f"GAME OVER - Distance: {self.distance} - Press R to restart",
            )
```

2. Replace `game/renderer.py`

This adds a visible blue shield outline around the helicopter while leaving the existing scene and obstacle rendering intact.

Python

```
"""
Renderer: all Pygame drawing lives here, kept separate from game logic.
"""

import pygame

WIDTH, HEIGHT = 700, 500
WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_BG = (140, 200, 230)
COLOR_HELI = (60, 60, 70)
COLOR_OBSTACLE = (70, 150, 80)
COLOR_TEXT = (20, 20, 20)
COLOR_SHIELD = (0, 100, 255)


def draw_scene(surface, helicopter, obstacles, shield_active=False):
```

```
    surface.fill(COLOR_BG)

    for obstacle in obstacles:
        pygame.draw.rect(surface, COLOR_OBSTACLE, obstacle.get_top_rect())
        pygame.draw.rect(surface, COLOR_OBSTACLE, obstacle.get_bottom_rect())

    heli_rect = helicopter.get_rect()

    if shield_active:
        # Draw the shield before the helicopter so the vehicle remains visible
        shield_rect = heli_rect.inflate(18, 18)
        pygame.draw.ellipse(
            surface,
            COLOR_SHIELD,
            shield_rect,
            width=3,
        )

    pygame.draw.rect(
        surface,
        COLOR_HELI,
        heli_rect,
        border_radius=4,
    )


def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)


def draw_banner(surface, font, text):
    surf = font.render(text, True, (180, 40, 40))
    rect = surf.get_rect(
        center=(
            surface.get_width() // 2,
            surface.get_height() // 2,
        )
    )
    surface.blit(surf, rect)
```

3. Test all four tasks

Run the game again:

```
PowerShell
python main.py
```

Check these cases:

1. **Shield activation:** Press `S`; the blue outline and `SHIELD: ON` should appear.
2. **Shield absorbs a hit:** Fly into an obstacle while protected. The shield should disappear, and the game should continue.
3. **Vulnerability returns:** Hit another obstacle without reactivating the shield. The game should end.
4. **Reactivation:** Press `S` during gameplay to turn the shield back on. Pressing `S` again turns it off manually.
5. **Restart:** After game over, press `R`. The game should restart with distance `0` and shield off.
6. **Regression checks:** Confirm movement and boundaries still work, collisions still end an unprotected run, and distance continues increasing during gameplay.

Important: Your `main.py` already sends `KEYDOWN` events to `engine.handle_keydown(event.key)`, so you should not need to change it for shield activation. The shield protects against obstacle collisions; it does not prevent the helicopter from reaching the screen boundaries.

> the shield turns off when i hit a wall but then i die straight after anyway. i think the same wall is hitting me again on the next frame. how do i fix that?

Exactly — your diagnosis is correct. The shield turns off after absorbing the first collision, but the helicopter is still overlapping the same wall. On the next frame, collision detection runs again, finds the same wall, and ends the game.

The fix: make the shield absorb the hit and remove the helicopter from the collision area by briefly moving the obstacle past the helicopter. The simplest reliable approach for your current game is to **remove the obstacle that hit the shield**.

Replace `_check_collisions()` in `game_engine.py`

Keep the rest of your file unchanged. Replace only the existing `_check_collisions()` method with this:

Python

```python
    def _check_collisions(self):
        helicopter_rect = self.helicopter.get_rect()
```

```
        for obstacle in self.obstacles:
            collided = (
                helicopter_rect.colliderect(obstacle.get_top_rect())
                or helicopter_rect.colliderect(obstacle.get_bottom_rect())
            )

            if not collided:
                continue

            if self.shield_active:
                # Absorb one hit, deactivate the shield, and remove
                # the obstacle so it cannot hit the helicopter again.
                self.shield_active = False
                self.obstacles.remove(obstacle)
                return

            # No shield: the collision ends the game.
            self.game_over = True
            return
```

Test it

1. Run `python main.py`.
2. Press **S** to activate the shield.
3. Hit a wall. The shield should turn off, but the game should continue.
4. Hit another wall without activating the shield again. The game should end.
5. Restart with **R** and confirm the shield starts off.

This works because the obstacle responsible for the shielded collision is removed from the obstacle list, so it cannot trigger another collision on the next frame.

when i press R the shield is still on from last game. make a new game always start with it off.

Your `reset()` method should set `self.shield_active = False`. That ensures the shield is off every time you press **R** to start a new game.

In `game_engine.py`, check that your `reset()` method looks like this:

‹/› **Python**

```
    def reset(self):
        """Start a fresh game."""
        self.helicopter = Helicopter(x=100, y=HEIGHT / 2)
        self.obstacles = []
        self.frames_until_spawn = 0
        self.game_over = False
        self.distance = 0
        self.shield_active = False
```

Also ensure `handle_keydown()` calls `self.reset()` when `R` is pressed after game over:

Python

```
    def handle_keydown(self, key):
        if self.game_over:
            if key == pygame.K_r:
                self.reset()
            return

        if key == pygame.K_s:
            self.shield_active = not self.shield_active
```

Why this fixes it: your `__init__()` calls `reset()` when the game starts, and pressing `R` calls the same method. Setting `self.shield_active = False` inside `reset()` guarantees the shield starts off both initially and after every restart.

Save the file, close the game window, and run `python main.py` again to test.

everything seems to work. using the expected behavior section of the readme, give me a quick checklist i can test by playing before i record the final video.

Worked for a few seconds

Here's a quick pre-recording checklist based directly on the README's **Expected Behavior** section. Play one run and tick each item off before recording your final 10-second video.

Lab 4 — Final gameplay checklist

Test progress

0 of 11 passed

Task 1 — Movement and boundaries

- [ ] Hold Up, then switch to Down: the helicopter reverses direction immediately.
- [ ] Hold Up and Down long enough to test both edges: the helicopter never leaves the screen.

Task 2 — Collision and game over

- [ ] Fly safely through an obstacle gap: the game continues.
- [ ] Touch a top or bottom wall without a shield: game over appears.

Task 3 — Distance scoring

- [ ] Distance increases automatically during gameplay.
- [ ] Game over displays the final distance, and it stops increasing.
- [ ] Press R: a new game starts with distance reset to 0.

Task 4 — One-hit shield

- [ ] Press S: the shield is visibly ON.
- [ ] Hit a wall with the shield active: it absorbs one hit, turns OFF, and the game continues.
- [ ] Hit another wall without reactivating the shield: game over occurs.
- [ ] Press R after game over: the new game starts with the shield OFF.

Mark all passed | Reset checklist

Before you record

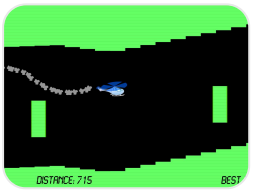
- Close and relaunch the game so you test the latest saved code.
- Make sure the game window and status text are clearly visible.
- Record a clean 10-second clip showing as much as possible: movement, increasing distance, and shield activation.
- If you can, capture a shield absorbing a hit; otherwise, demonstrate that behavior during your testing before recording.

Important: A single 10-second clip may not show every feature convincingly. Prioritize a clear demonstration of the new functionality, and make sure all checklist items work before submitting.

what should i show in the 10 second after video so every fix and feature is visible?

The best strategy is to record a **single continuous 10-second gameplay clip** that prioritizes the new features while showing the movement fix naturally. Realistically, all fixes cannot be demonstrated conclusively in 10 seconds, especially the restart and both boundaries, so test those separately before recording.

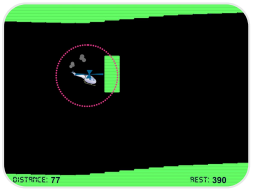
Recommended 10-second recording plan



0–2 seconds — Movement and score

Play normally and briefly switch between Up and Down. Keep the distance counter visible as it increases.

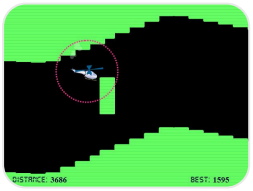
Task 1 + Task 3



2–5 seconds — Shield activation

Press **S** and show the blue shield outline and `SHIELD: ON` status while moving toward an obstacle.

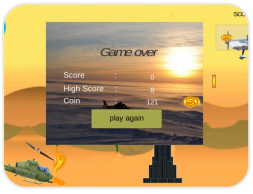
Task 4



5–8 seconds — Shield absorbs a hit

If you can time it, collide with a wall while shielded. Show the shield switching off while gameplay continues and distance remains visible.

Task 2 + Task 4



8–10 seconds — Game over and final score

If the timing works, hit another wall without the shield and show the game-over message with the final distance.

Task 2 + Task 3

What about the remaining requirements?

Before recording, separately verify these because squeezing them into the same clip would make the demonstration rushed:

- Both top and bottom boundaries keep the helicopter on-screen.
- Flying through a gap does not trigger game over.
- Pressing **R** resets the distance to `0` and the shield to OFF.

My recommendation: prioritize the shield absorbing a hit, its deactivation, and the final score/game-over screen in your video. Those are the most distinctive new behaviors. Keep the clip continuous and unedited if possible, and don't worry if you cannot demonstrate every edge case in 10 seconds.

how do i push everything to my github repo vivekvarma006/SE-Lab, and how do i make sure nothing went to the SETAPESU26 repo?

We'll push your completed Lab 4 to your personal repository, `vivekvarma006/SE-Lab`, and verify the destination before sending anything.

The most important rule: **never run `git push` until you've checked the push URL.** Git's `origin` remote determines where a normal push goes. You can verify and change that URL using Git's remote commands.

GitHub Docs +1

Step 1 — Open your existing repository

Open PowerShell and run:

PowerShell

```
cd "$HOME\Downloads\SE-Lab"
```

Confirm that you're in the right Git repository:

PowerShell

```
git status
git branch --show-current
git log --oneline -8
```

Your earlier Task 1 commit should appear in the recent history.

Step 2 — Check where Git will push

Run:

```
PowerShell
git remote -v
```

Look at both the `(fetch)` and `(push)` URLs.

Your intended destination is:

`https://github.com/vivekvarma006/SE-Lab.git`

If `origin` already points to your personal repo

Proceed to Step 3.

If `origin` points to `SETAPESU26/08_helicopter` or any other repository

Do not push yet. Set the `origin` URL to your personal repo:

```
PowerShell
git remote set-url origin https://github.com/vivekvarma006/SE-Lab.git
```

Now verify it again:

```
PowerShell
git remote -v
```

Both the fetch and push URLs for `origin` should now point to `vivekvarma006/SE-Lab`.

If you see another remote named `upstream` pointing to `SETAPESU26`, that's okay by itself. Just don't push to it.

Step 3 — Check and commit your Lab 4 files

Check what's changed:

```
PowerShell
git status
```

If your Task 2–4 changes are still uncommitted, inspect the files:

PowerShell

```
git diff
```

Then stage the Lab 4 folder:

PowerShell

```
git add Lab-4
```

Review exactly what is staged:

PowerShell

```
git status
git diff --cached --stat
```

If the staged files are correct and contain no accidental videos, temporary files, or unrelated changes, commit them:

PowerShell

```
git commit -m "Complete Lab 4 helicopter game features"
```

Important: Your lab instructions ask for separate commits for each task. You already made a Task 1 commit earlier. If Tasks 2, 3 and 4 are still combined in your working changes, this command creates one combined commit for those remaining changes. For strict per-task commit history, they would need to be committed separately.

If Git says `nothing to commit`, your work may already be committed. Check `git log --oneline -8` before doing anything else.

Step 4 — Push only to your personal repo

First, verify the destination one final time:

PowerShell

```
git remote get-url --push origin
```

It must print:

```
https://github.com/vivekvarma006/SE-Lab.git
```

Then check your current branch:

PowerShell

```
git branch --show-current
```

If the branch is `main`, push using:

PowerShell

```
git push -u origin main
```

If it prints a different branch name, use that exact name instead of `main`. For example, if it prints `master`, run `git push -u origin master`.

If Git asks you to authenticate, follow the GitHub sign-in prompt.

Step 5 — Verify everything on GitHub

Open your personal repository:

Open vivekvarma006/SE-Lab on GitHub ↗

Check these items:

- [ ] `Lab-4` is visible in the repository.
- [ ] It contains your updated game code and the required deliverables.
- [ ] The latest commits appear under your personal repository's commit history.
- [ ] The latest code on GitHub matches your local code.

Finally, run:

PowerShell

```
git status
git log --oneline -5
git remote -v
```

A clean working tree should say `nothing to commit, working tree clean`.

How to be certain you haven't pushed to SETAPESU26

The key checks are:

1. `git remote get-url --push origin` shows `vivekvarma006/SE-Lab`.
2. Your push command explicitly uses `origin`, not `upstream`.
3. You can see the new commits in your personal repository on GitHub.

Changing your local remote URL does not itself modify the assigned repository. And simply having an `upstream` remote pointing to `SETAPESU26` does not push anything there.

Do this now: Run Steps 1 and 2, then paste the output of `git status`, `git branch --show-current`, and `git remote -v` here. I'll help you verify the destination and choose the exact safe push command before you execute it.

how do i export this whole chat as a pdf? the readme also asks for the chat link.

Your README asks for two separate deliverables: **the complete chat history exported as a PDF/doc and the Chat/LLM page link**. The safest approach is to provide both.

1. Export this conversation as a PDF

Since you're using ChatGPT in a desktop browser, the easiest method to try is printing the conversation to PDF.

Step 1 — Open the full conversation

Stay in this chat and scroll through the conversation to ensure the earlier messages have loaded.

Step 2 — Open the print dialog

Press **Ctrl + P** on Windows.

Step 3 — Save as PDF

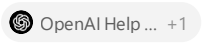
In the print dialog, set the destination to **Save as PDF**, then click **Save** and use a filename such as `Lab-4_Chat_History.pdf`.

Important: Browser printing may omit older messages, format code blocks poorly, or include only the currently rendered part of a long conversation. Check the resulting PDF before submitting it. If it is incomplete, use ChatGPT's data export or copy the conversation into a document and export that as PDF.

2. Get the chat link required by the README

1. Open this conversation in ChatGPT.
2. Click **Share** near the top-right of the conversation. If you don't see it, check the conversation menu in the sidebar.

3. Review the preview to ensure it includes the full conversation, not just an individual response.
4. Click **Create link** or **Copy link**, depending on what appears.
5. Open the copied link in a new tab to verify it works and displays the chat history.



Privacy note: Anyone who has the shared link may be able to view the shared conversation. Review it before sharing.

3. Put both deliverables in `Lab-4`

Your final folder should look something like this:

SE-Lab/Lab-4/

- `before.mp4`
- `after.mp4`
- `Lab-4_Chat_History.pdf`
- `main.py`, `requirements.txt`, `README.md`
- `game/` — all updated Python files
- `CHAT_LINK.txt` — paste your copied ChatGPT share URL here

Save the link in `CHAT_LINK.txt` so the evaluator can find it easily. You can also include it in your submission notes.

One last check: Open the PDF and shared link yourself. Make sure the PDF includes the relevant prompts and responses, and the link displays the conversation history. Then commit and push the deliverables to your personal `vivekvarma006/SE-Lab` repository.